

PROJECT DRAFT

TITLE: HR Management application.

Repository : <https://github.com/NisNisSama/HRManagementProject>

OVERVIEW:

The HR management application is an application designed to manage human resource data and processes. It has as objectives :

- Store and manage information about employees
- Automate HR processes as leaves, payroll and recruitment.
- Track performance and records for the HR department

MODULES :

- Employee : information, documents.
- Payroll : salary, taxes, bonus, deduction.
- Attendance : daily attendance, leave (paid), holiday (unpaid).
- Recruitment : recruitment announcement, resume database, recruitment progression.
- User management : admin, permission and access.

FUNCTIONALITIES and ACCESS:

Employee Module:

- Employee information entry : HR
- Employee information modification : HR, EMP
- Employee information deletion : ADMIN
- Document uploading : EMP
- Document update : EMP
- Document deletion : ADMIN

Payroll Module:

- Monthly salary budget report : HR overall report, HR EMP personal report
- Net Salary calculation
- Salary updating : HR
- Bonus assignation : HR

Attendance Module:

- Daily attendance sign-in : EMP
- Monthly attendance report : HR overall report, HR EMP personal report

- Leave request : EMP
- Holiday request : EMP
- Leave acceptance : HR
- Holiday acceptance : HR

Recruitment Module:

- Job listing : HR
- Job update : HR
- Job application : Public
- Resume list : HR
- Candidate selection : HR
- Recruitment progression update : HR

User Management Module:

- Roles assignation : ADMIN
- Password modification : ADMIN, EMP
- Login : EMP

TEST :

Each functionalities shall be tested one by one to ensure correct and smooth operation of the application. Listing but not limited to the following:

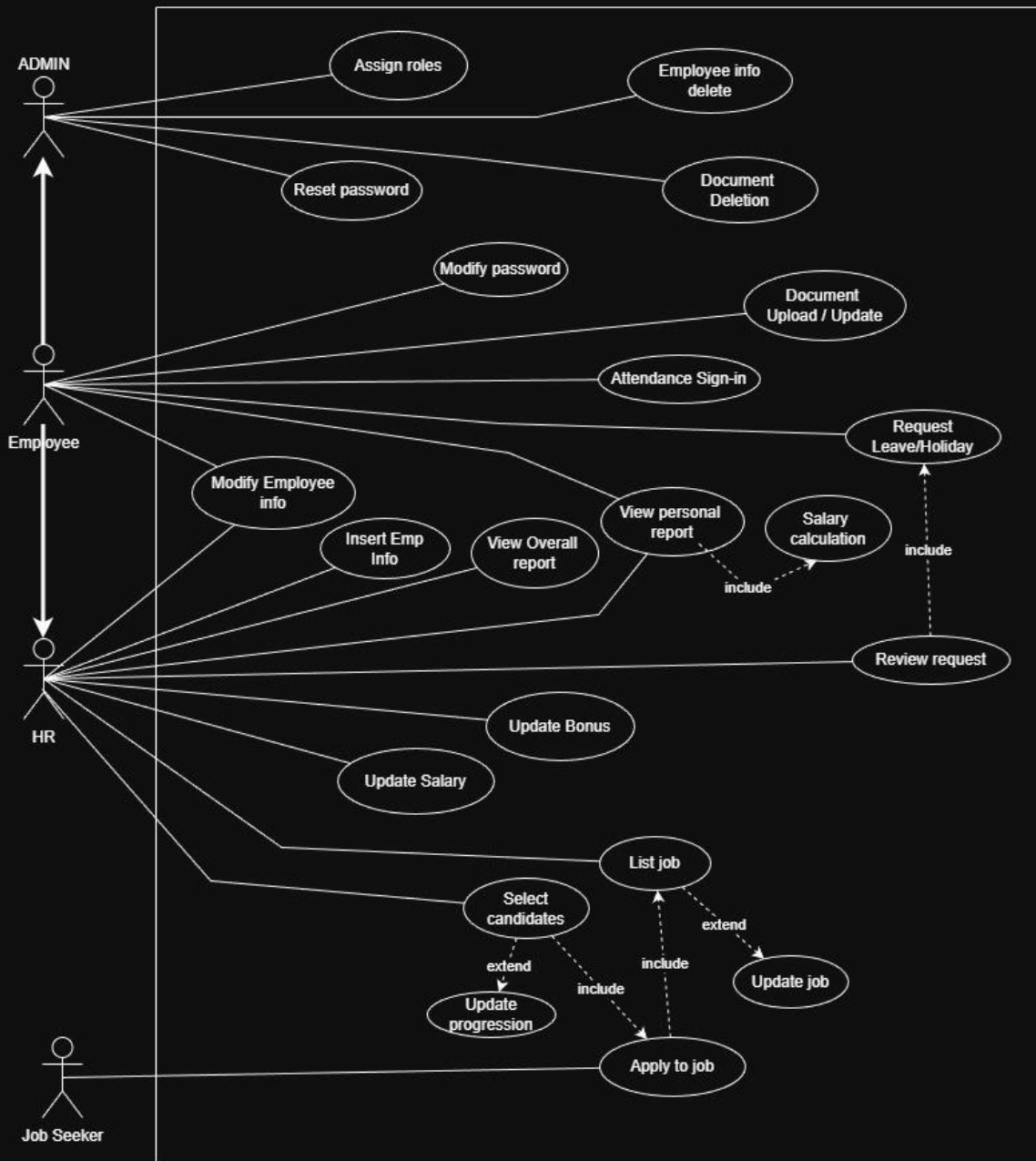
- Employee : login, CRUD, file upload.
- Payroll : salary calculation, edge cases.
- Attendance: daily track, leave calculation, conflict leave test.
- Recruitment : job posting creation, application submission, status tracking
- User and role management : login, logout, role based access test, permission escalation prevention.

SECURITY:

- Authentication: login
- Access management: roles and admin
- Input validation
- File format checking
- Scripting prevention

USE CASE DIAGRAM:

HR MANAGEMENT SYSTEM

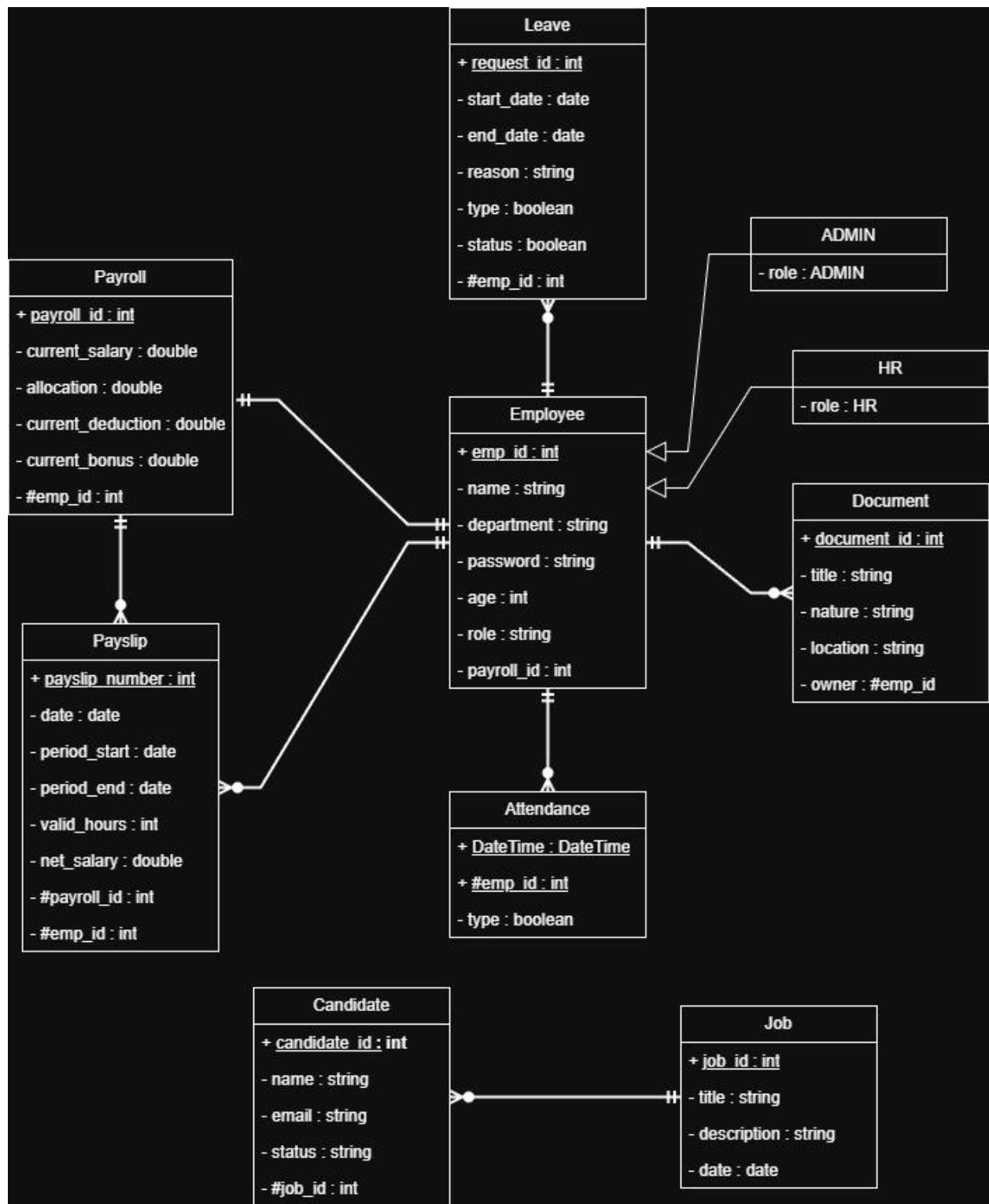


CLASS DIAGRAM (PROTOTYPE) :

Data Dictionary:

- Employee:
int emp_id(primary_key), **string** name, **string** department, **string** password, **int** age, **char** gender, **string** role, **int** payroll_id(FK from payroll)
- Document:
int document_id(primary_key), **string** title, **string** nature, **string** location, **int** owner (foreign_key from employee)
- Attendance:
int DateTime+#emp_id(primary_key), **bool** type(0=out, 1=in)
- Leave:
int request_id(primary_key), **date** start_date, **date** end_date, **string** reason, **bool** type (1 = Paid, 0 =Unpaid), **bool** status (0=declined, 1=accepted, empty=pending), #emp_id
- Payroll:
int payroll_id(PK), **double** current_salary, **double** allocation, **double** current_deduction, **double** current_bonus, #emp_id
- Payslip:
int payslip_number(PK), **date** date, **date** period_start, **date** period_end, **int** valid_hours, **double** net_salary, #payroll, #emp_id
- Job: **int** job_id(PK), **string** title, **string** description, **date** date
- Candidate: **int** candidate_id(PK), **string** name, **string** email, **string** status, #job_id

Diagram:



SQL Script to scheme the database:

```
-- =====
-- 1. INDEPENDENT TABLES (No Foreign Keys)
-- =====

CREATE TABLE Job (
    job_id INT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    date DATE NOT NULL
);

-- =====
-- 2. CORE ENTITIES (Referencing Independent Tables)
-- =====

CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    department VARCHAR(255) NOT NULL,
    password VARCHAR(255) NOT NULL, -- Ensure this is long enough for hashing
    age INT,
    gender CHAR(1),
    role VARCHAR(50),
    payroll_id INT -- Kept per diagram, nullable to avoid circular insert
deadlocks
);

CREATE TABLE Candidate (
    candidate_id INT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    status VARCHAR(50) NOT NULL,
    job_id INT NOT NULL,
    FOREIGN KEY (job_id) REFERENCES Job(job_id) ON DELETE CASCADE
);

-- =====
-- 3. EMPLOYEE DEPENDENT TABLES
-- =====

CREATE TABLE Leave (
    request_id INT PRIMARY KEY,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    reason TEXT,
    type BOOLEAN NOT NULL, -- 1 = Paid, 0 = Unpaid
    status BOOLEAN, -- 0 = Declined, 1 = Accepted, NULL = Pending
    emp_id INT NOT NULL,
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id) ON DELETE CASCADE
);

CREATE TABLE Document (
    document_id INT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    nature VARCHAR(100),
```

```

        location VARCHAR(512),    -- Path or URL to file storage
        owner INT NOT NULL,       -- #emp_id mapping
        FOREIGN KEY (owner) REFERENCES Employee(emp_id) ON DELETE CASCADE
    );

CREATE TABLE Attendance (
    clock_time TIMESTAMP,        -- Maps to your 'DateTime' type
    emp_id INT,
    type BOOLEAN NOT NULL,       -- 0 = Out, 1 = In
    PRIMARY KEY (clock_time, emp_id), -- Composite Primary Key
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id) ON DELETE CASCADE
);

-- =====
-- 4. PAYROLL & FINANCIAL ENTITIES
-- =====

CREATE TABLE Payroll (
    payroll_id INT PRIMARY KEY,
    current_salary DOUBLE PRECISION NOT NULL,
    allocation DOUBLE PRECISION DEFAULT 0.0,
    current_deduction DOUBLE PRECISION DEFAULT 0.0,
    current_bonus DOUBLE PRECISION DEFAULT 0.0,
    emp_id INT NOT NULL,
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id) ON DELETE CASCADE
);

CREATE TABLE Payslip (
    payslip_number INT PRIMARY KEY,
    date DATE NOT NULL,
    period_start DATE NOT NULL,
    period_end DATE NOT NULL,
    valid_hours INT NOT NULL,
    net_salary DOUBLE PRECISION NOT NULL,
    payroll_id INT NOT NULL,
    emp_id INT NOT NULL,
    FOREIGN KEY (payroll_id) REFERENCES Payroll(payroll_id) ON DELETE CASCADE,
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id) ON DELETE CASCADE
);

-- =====
-- 5. RESOLVING THE CIRCULAR RELATIONSHIP
-- =====
-- Because Employee references Payroll and Payroll references Employee,
-- we add the Foreign Key constraint to Employee *after* both tables exist.

ALTER TABLE Employee
ADD CONSTRAINT fk_employee_payroll
FOREIGN KEY (payroll_id) REFERENCES Payroll(payroll_id) ON DELETE SET NULL;

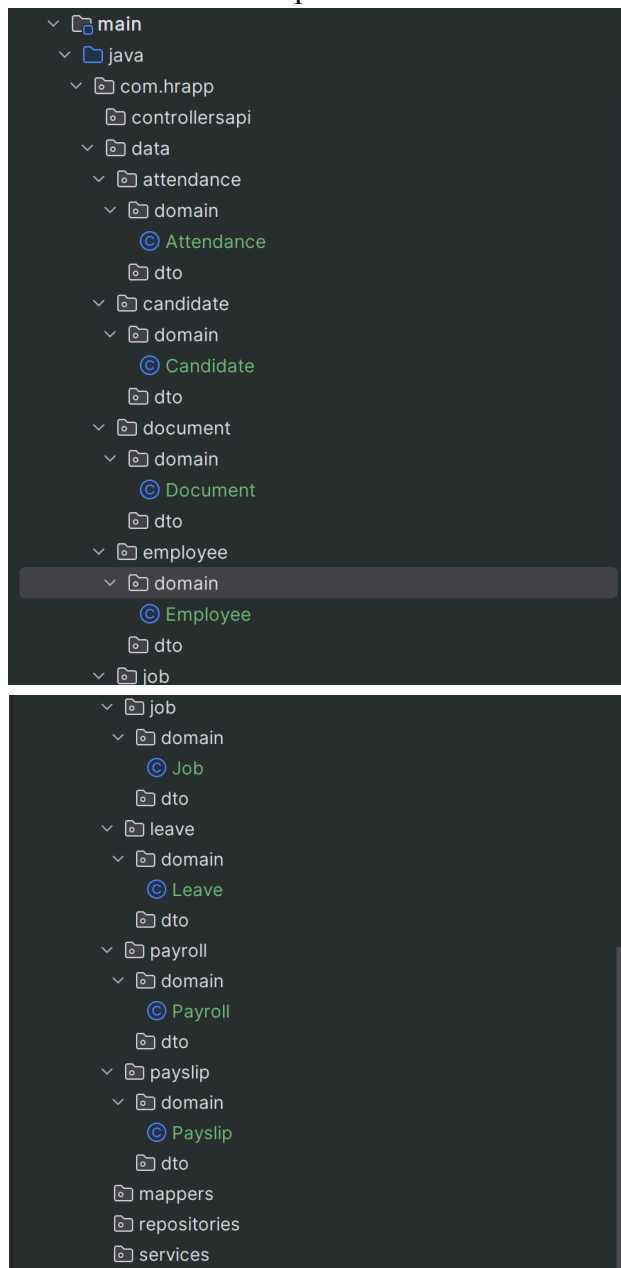
```

IMPLEMENTATION WITH MICRONAUT

Domains definition:

We will map the tables above to our micronaut application. It is important to note that this is only the representation of the database in our application, the payload used for the api calls will be under the DTO section.

Here is our folder structure after the implementation of domains



Notice that our application directories are separated following Micronaut flow logic. As DTO and Domain contain no business logic or any processing, they are just pure data that will be manipulated by the application. This is why it is isolated from the rest of the logic implementation.